



# End-to-End Navigation II

Graduate Course INTR-6000P

Week 12 - Lecture 24

**Changhao Chen**

Assistant Professor

HKUST (GZ)

# End-to-End Self-Driving

2014–2018: Modular + HD map

2019–2021: Sensor fusion + planning

2022–2023: BEV + multi-task learning

2024→ : End-to-end Navigation + VLM grounding



# End-to-End Self-Driving

## Pipeline Overview:

**Sensors → BEV → Perception → Behavior prediction → Trajectory → Control**

Modules include:

- BEV mapping
- Object & lane detection
- Agent intent prediction
- Trajectory head
- Safety controller

| Choice         | Options                                       |
|----------------|-----------------------------------------------|
| Input          | Camera / LiDAR / IMU / GPS                    |
| Representation | Raw pixels / BEV / voxel grid / 4D attention  |
| Output         | Control / trajectory / affordances            |
| Training       | IL, RL, self-supervision                      |
| World Model    | Transformer temporal memory, diffusion models |

# End-to-End Self-Driving

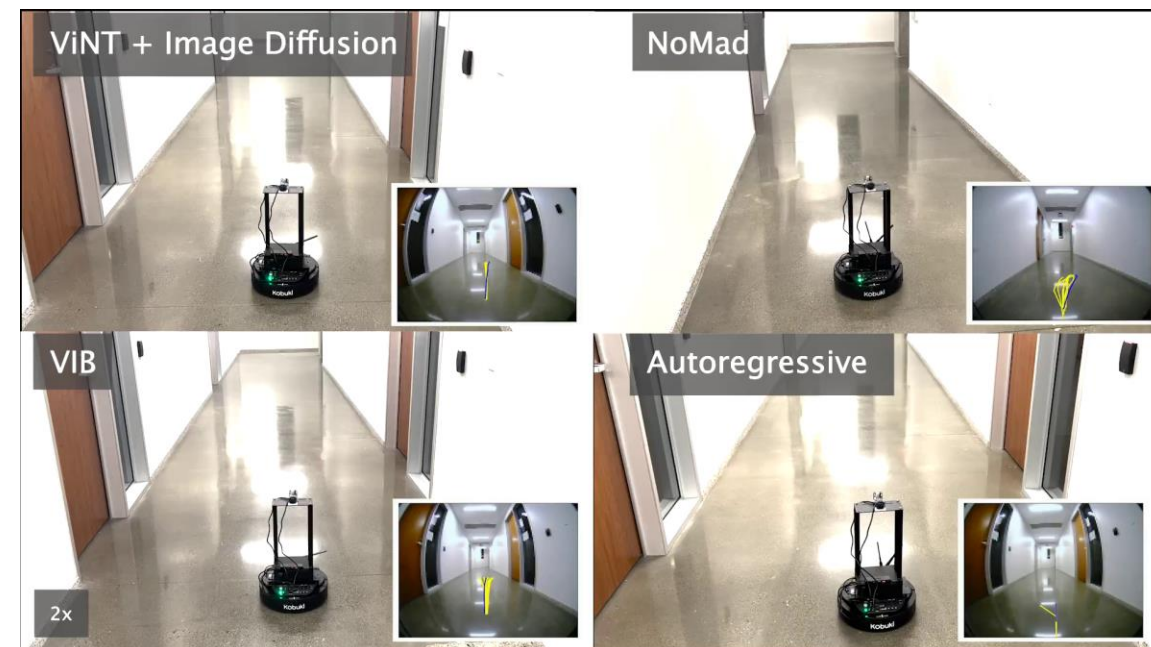
## Why End-to-End for Driving?

### Conventional stack issues:

- Manual feature engineering brittle
- HD maps expensive & degrade
- Strong human priors limit generalization

### End-to-end advantages:

- Joint learning = fewer failure cascades
- Learning from real driving behavior
- Better long-tail behavior & emergent priors



# Typical End-to-End Self-Driving Models

## Direct Control Prediction

Core Concept: A single deep learning model maps raw sensor data (e.g., images) directly to vehicle control commands (steering, throttle, brake).

Exemplar Models: PilotNet (NVIDIA), CILRS (Wayve), LAV (UC Berkeley).

Key Advantage: Ultra-fast inference, enabling real-time reaction.

Primary Challenge: Limited interpretability, creating a "black box" problem for validation and debugging.

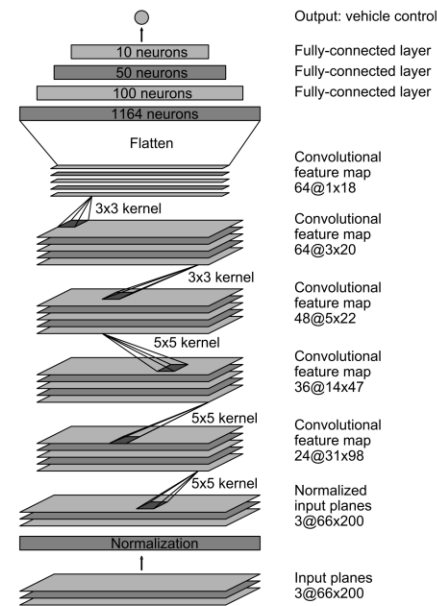
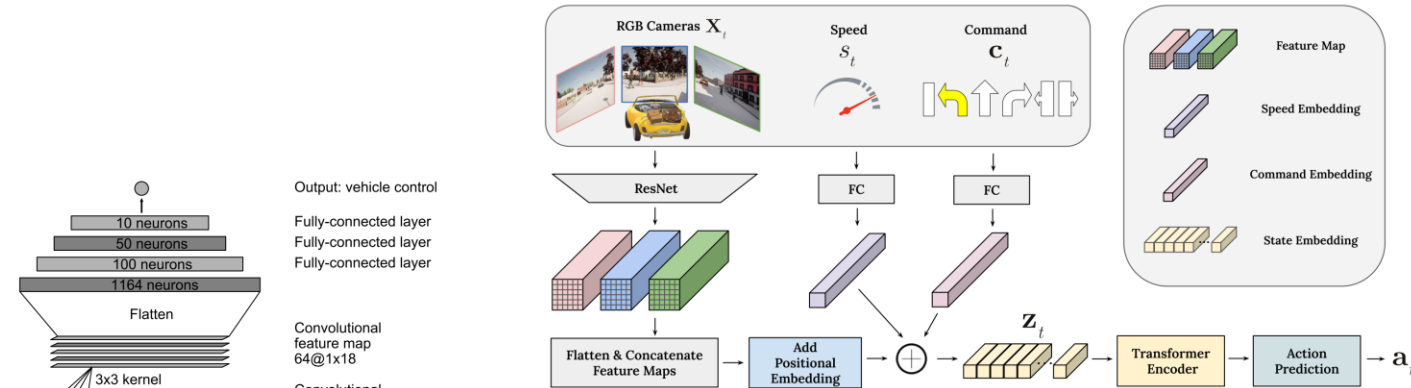
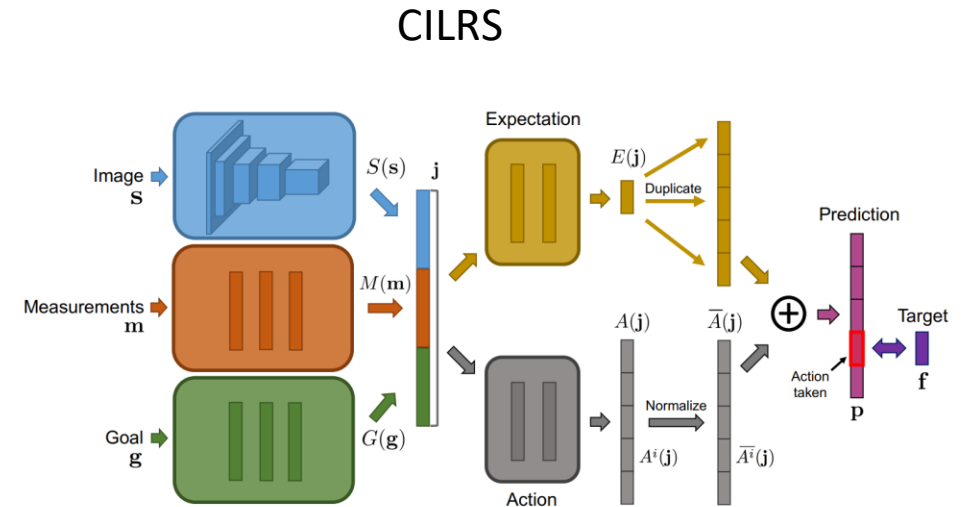


Figure 1: PilotNet architecture.

## PilotNet



## LAV



# Typical End-to-End Self-Driving Models

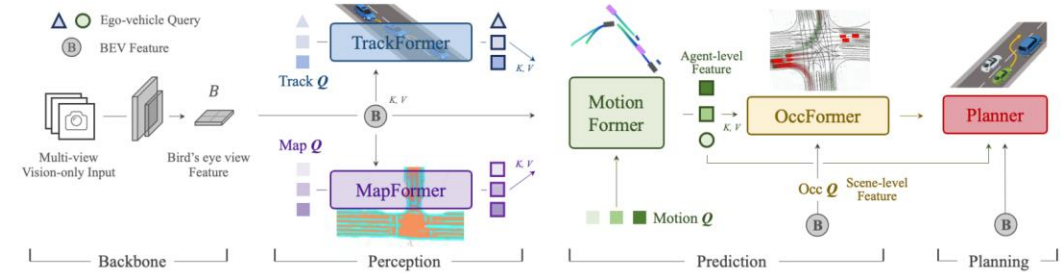
## Intermediate Representation-Driven Models

Exemplar Models: UniAD, VAD, TCP

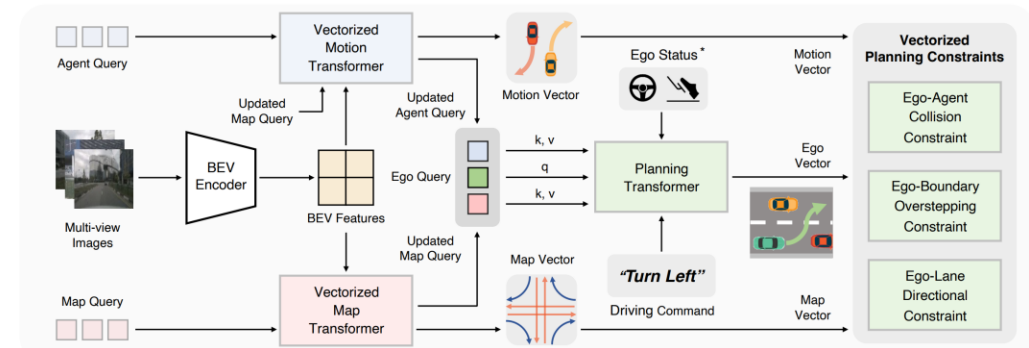
Core Innovation: Learns a rich, internal Bird's-Eye View (BEV) or semantic map as a foundational world model.

Structured Workflow:

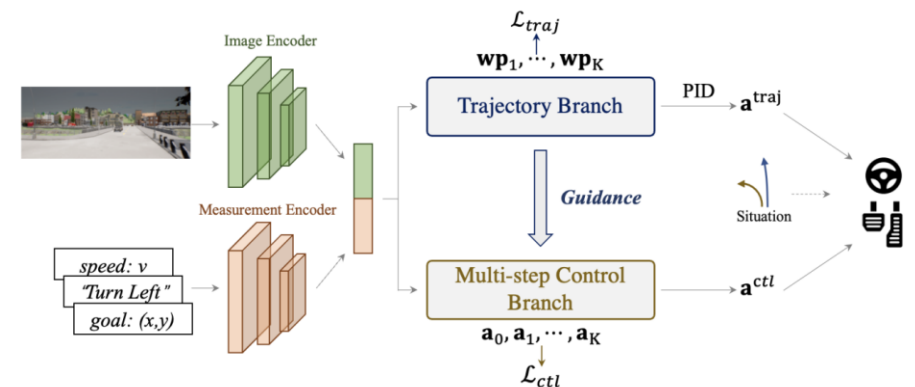
- Perception: Fuses sensor data to construct a dynamic BEV map, encoding the geometry and semantics of the scene.
- Planning: Uses this rich representation to predict others' trajectories and plan a safe, comfortable, and compliant ego-path.
- Control: Executes the planned trajectory with smooth and precise vehicle control.



UniAD



VAD

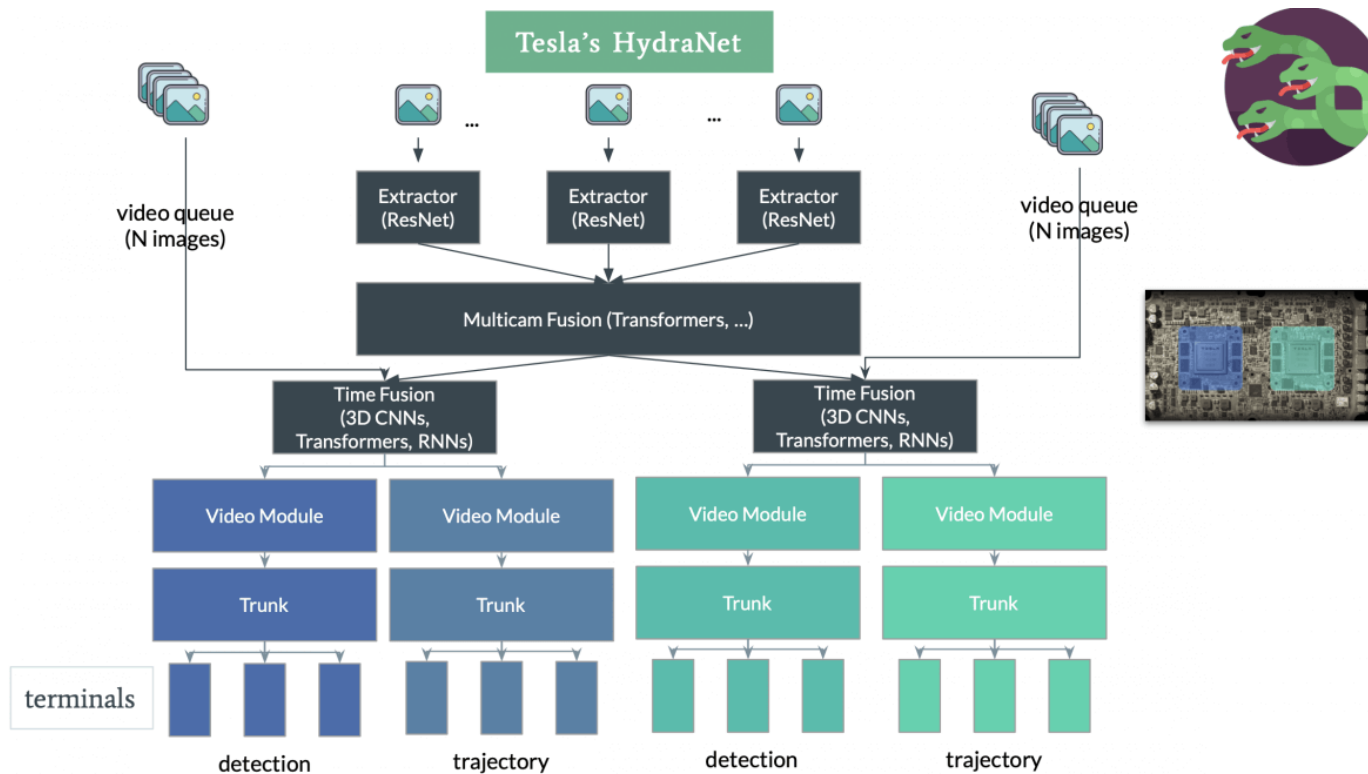


TCPNet

# Case Study: Tesla FSD (Vision-only)

## Tesla FSD: Fully end-to-end systems (2021):

"It's like Chat-GPT, but for cars!!!" "Instead of determining the proper path of the car based on rules, we determine the car's proper path by using neural network that learns from millions of training examples of what humans have done."

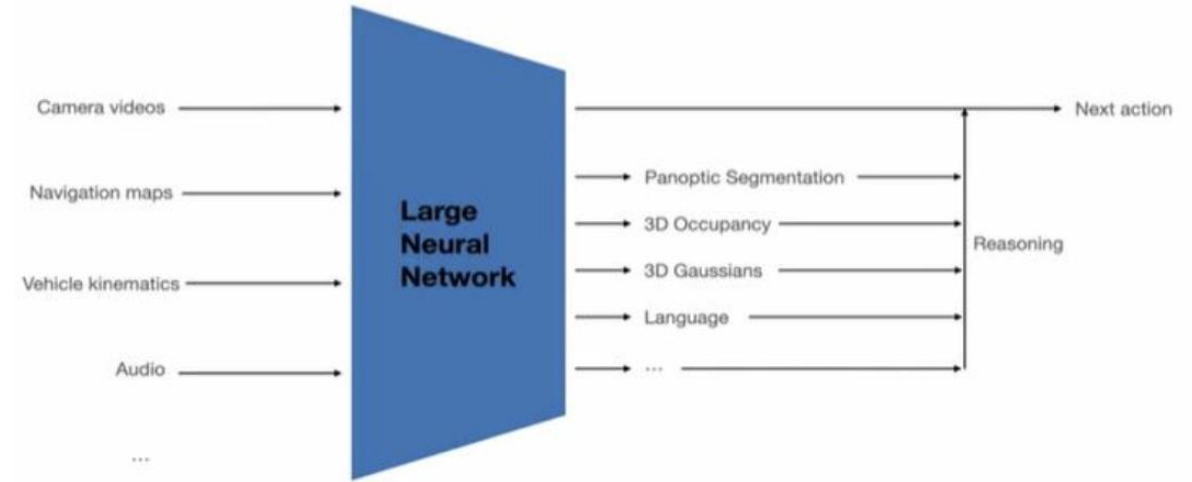


# Case Study: Tesla FSD (Vision-only)

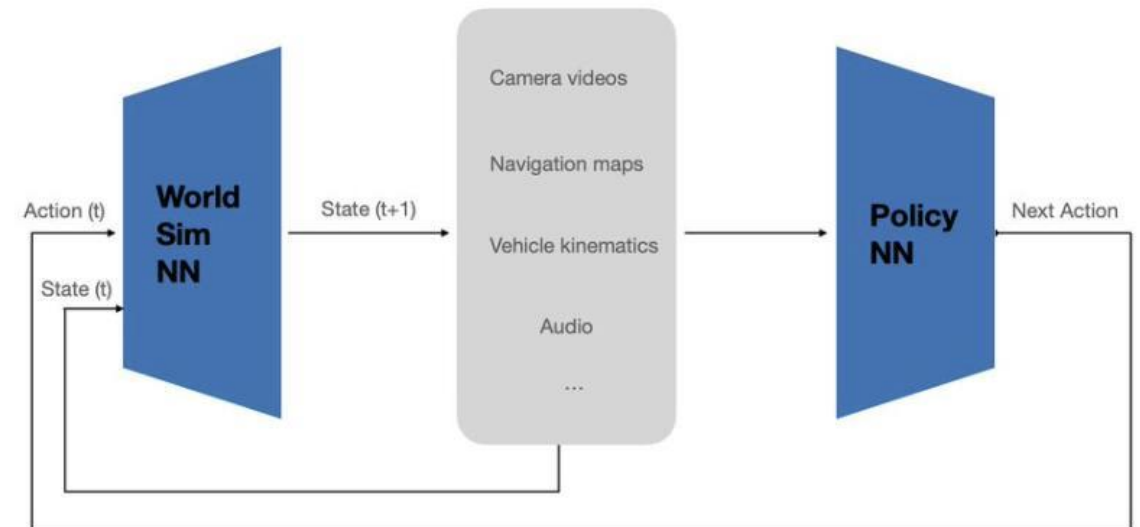
## Tesla FSD: Fully end-to-end systems (2025)

Main Challenges of learning pixels -> control

1. Curse of dimensionality
2. Interpretability and safety guarantees
3. Evaluation



Interpretability and safety guarantees



Neural Network closed-loop simulator Can be trained with cheap to collect state-action data



# Case Study: Wayve

Our innovative approach replaces the modular 'sense-plan-act' architecture of the traditional AV1.0 approach with a single neural network trained on diverse data to convert raw sensor inputs into safe driving outputs.

**Driving Input,  $10^8$  dimensions**



Cameras (6 @ 25 Hz)



GNSS



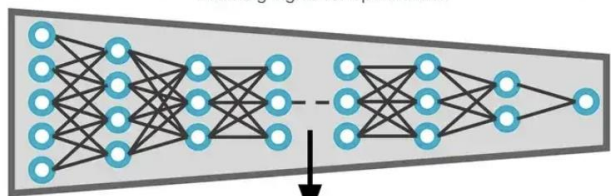
Basic Sat-nav Map



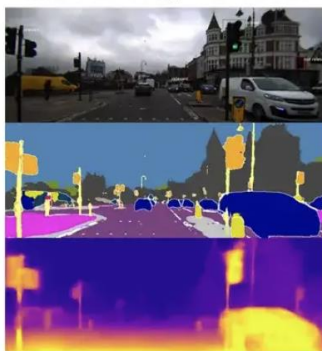
Vehicle State

+ other sensing modalities  
where required, e.g. RADAR

Representation signal  
Learning signal for optimisation



Decoded human-interpretable  
intermediate representations



Semantics, geometry, motion prediction.

**Driving Output,  $10^1$  dimensions**

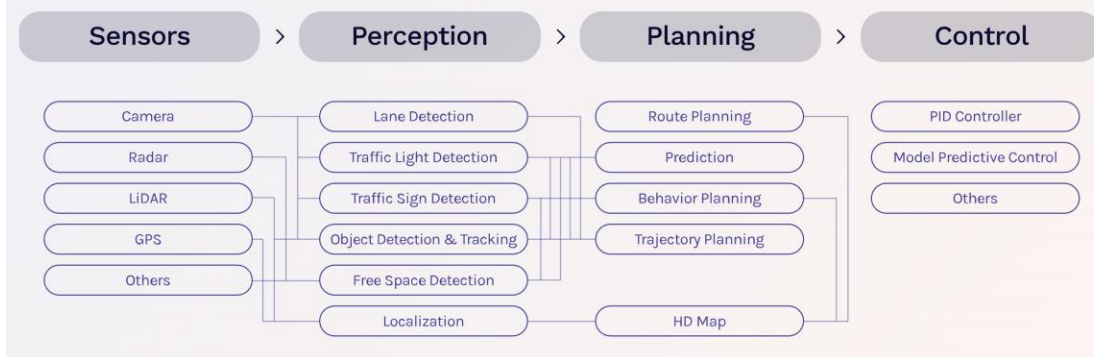


Motion Plan

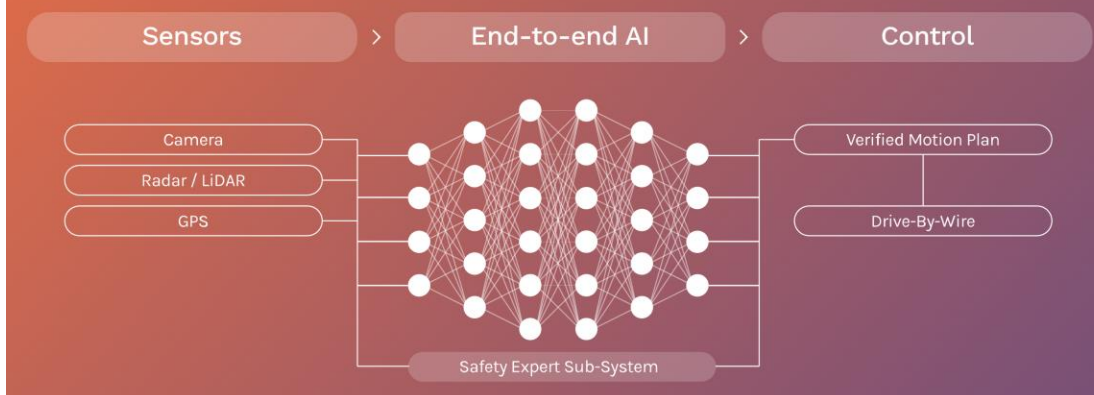


Vehicle Controls

**AV1.0**



**AV2.0**



# Safety & Reliability

## Key challenges:

- Corner cases
- Uncertainty calibration
- Formal safety layers
- Rule grounding & driver norms
- Adversarial robustness
- Regulatory requirements

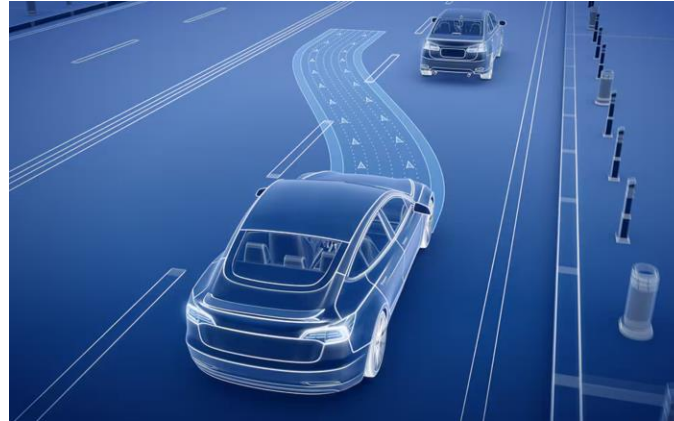
## Takeaways :

- E2E does not mean “black-box”
- BEV + temporal transformers dominate
- Industry uses hybrid safety stack
- Learned planning → improved generalization

# Embodied Navigation Beyond Cars

## Platforms:

- Indoor service robots
- Warehouse AMRs
- Delivery robots (sidewalk bots)
- Drones (inspection, racing)
- Home humanoids



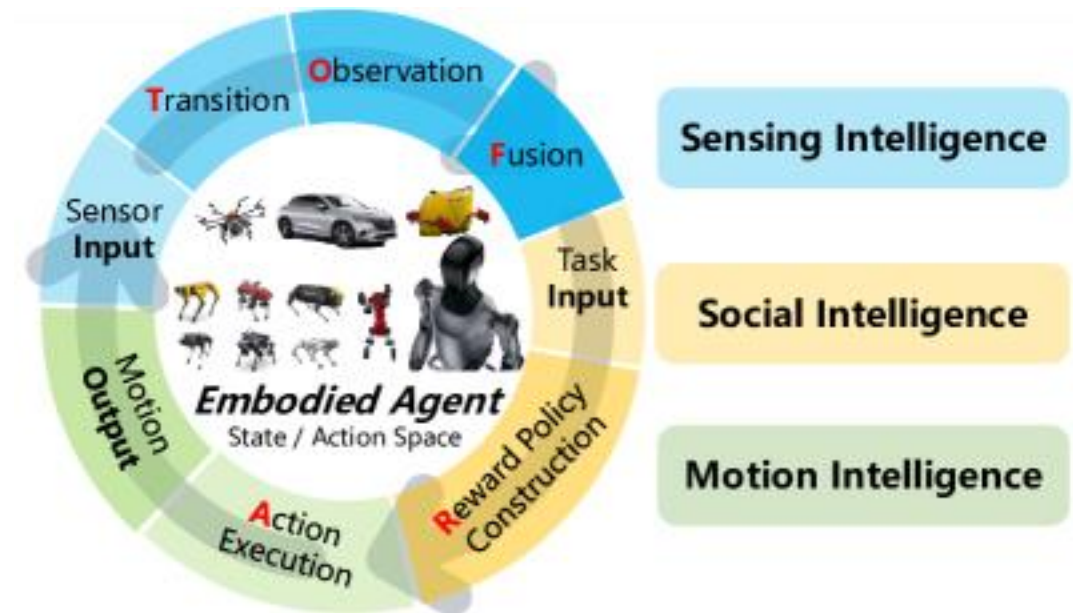
# Embodied Navigation Loop

## Pipeline:

Observe → Understand → Navigate → Interact

## Sensors:

- RGB, depth, stereo
- LiDAR, IMU, wheel odometry
- Language instructions (VLM era)
- Tactile for manipulation robots



Source: Sensing, Social, and Motion Intelligence in Embodied Navigation: A Comprehensive Survey

# End-to-End Robot Navigation

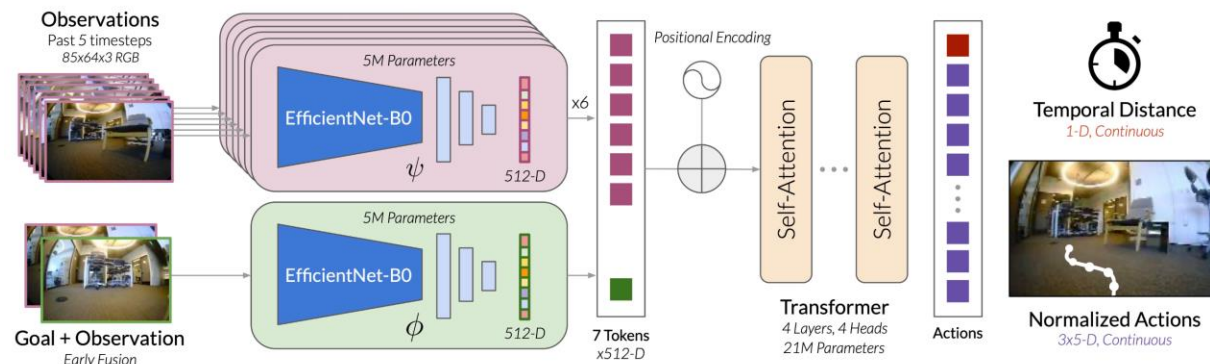
## Vision based Robot Navigation Model:

ViNT: A foundation model for visual navigation

Architecture: Transformer-based, pre-trained on large-scale, heterogeneous robot data.

Mechanism: Goal-conditioned sequence prediction. Encodes a history of images and a goal image to autoregressively predict velocity commands.

Strength: Demonstrates strong few-shot adaptation and generalization to new robots and environments.





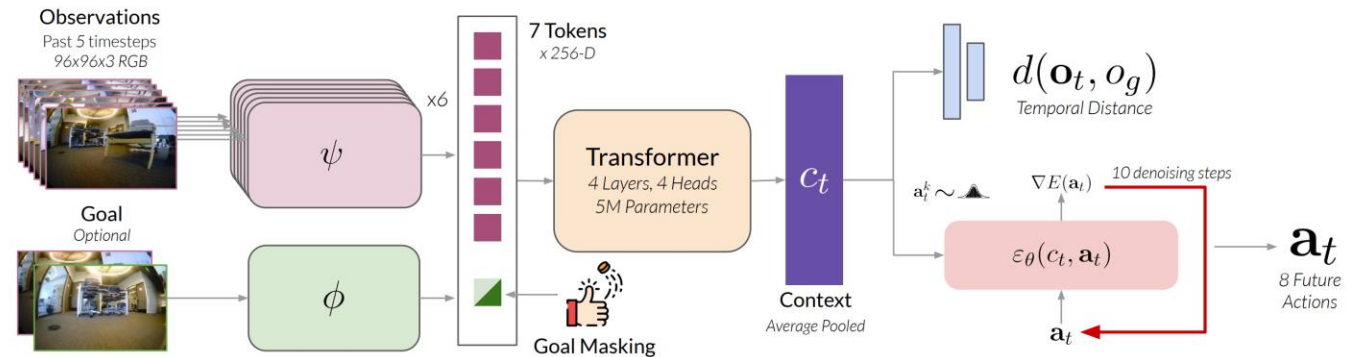
# End-to-End Robot Navigation

## Vision based Robot Navigation Model:

### NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration

Imitation learning model that uses diffusion policies to generate robust and diverse navigation behaviors.

Core Idea: Employs a masked goal-conditioning strategy and normalized data to handle multi-modal scenarios and improve generalization.



# End-to-End Robot Navigation

## Vision-Language-Action (VLA) Agents

Agents that map:

**(Images,Language)→Navigation+Manipulation**

Examples:

- SayCan / RT-1 / RT-2
- PaLM-E
- MobileVLA / Embodied-GPT

# End-to-End Robot Navigation

## Vision-Language Navigation (VLN)

Task examples:

- “Go to the kitchen and find the refrigerator.”
- “Navigate to the red sofa and open the drawer.”

Benchmarks:

- R2R, RxR
- ALFRED / Ego-VQA
- Habitat-VLN
- BEHAVIOR-1K

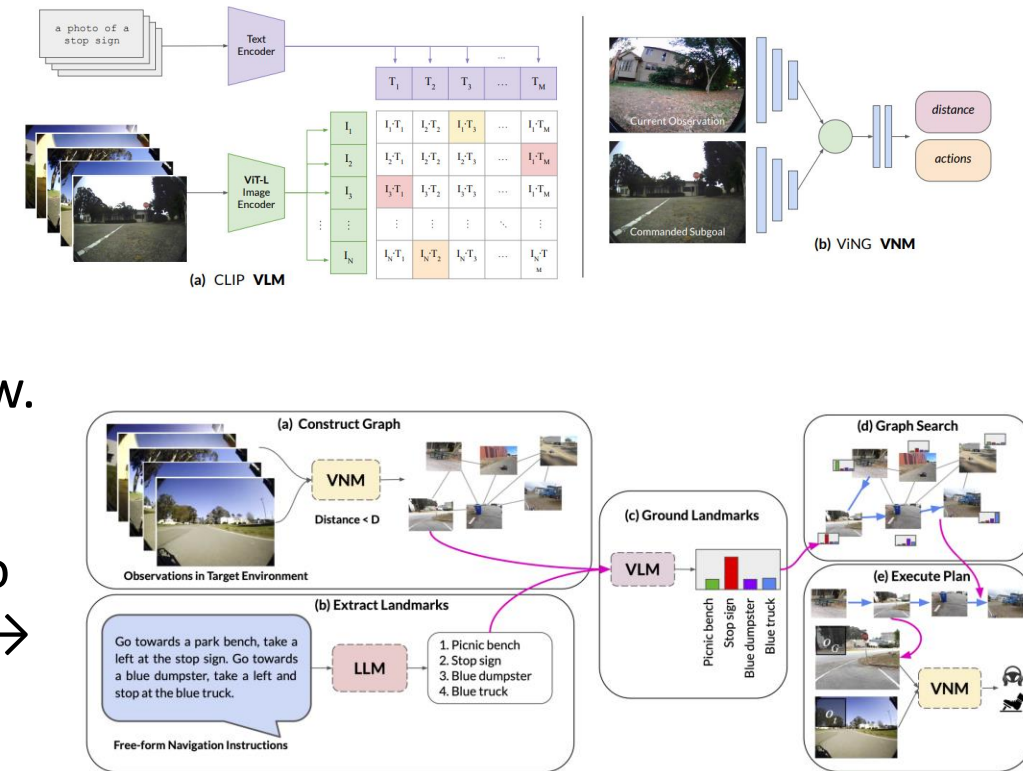
# End-to-End Robot Navigation

## Typical VLN Model:

### LM-Nav:

Concept: Uses a large language model (LLM) as a semantic planner. It translates high-level natural language commands ("get my coffee from the kitchen") into a sequence of visual landmarks to follow.

Flow: Language  $\rightarrow$  LLM  $\rightarrow$  Landmark Sequence ("go to the hallway, then the kitchen")  $\rightarrow$  Visuomotor Policy  $\rightarrow$  Actions.



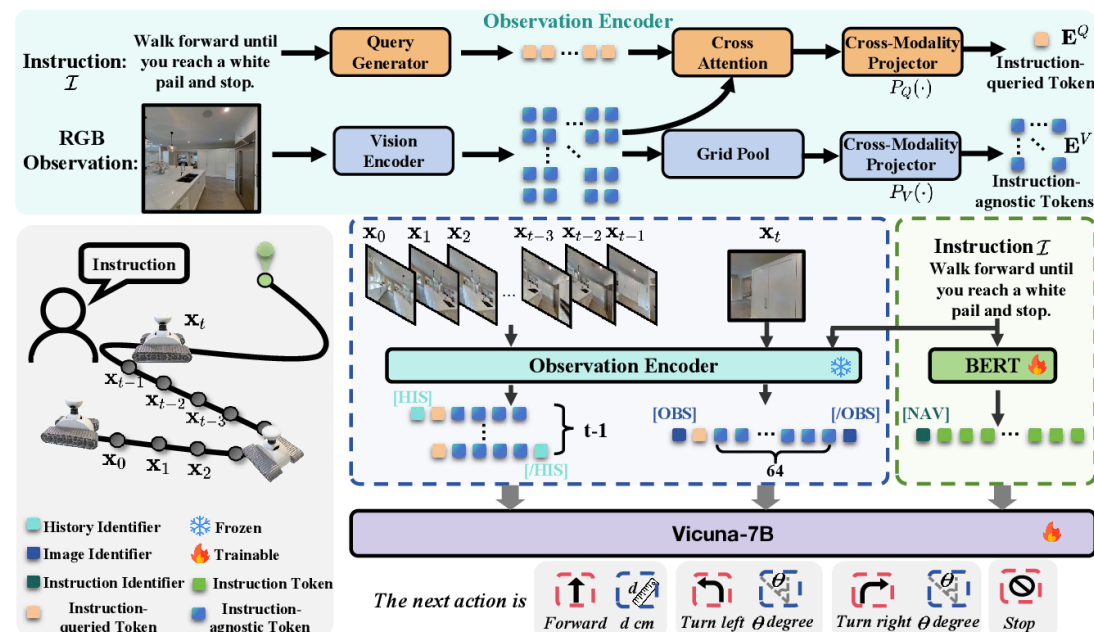
# End-to-End Robot Navigation

## Typical VLN Model:

Navid:

Concept: An LLM-powered multimodal agent that controls a robot directly from instructions and visual input. It reasons about actions in real-time.

Flow: Continuously integrates language instruction, camera images, and its own actions to decide the next move, enabling closed-loop control.

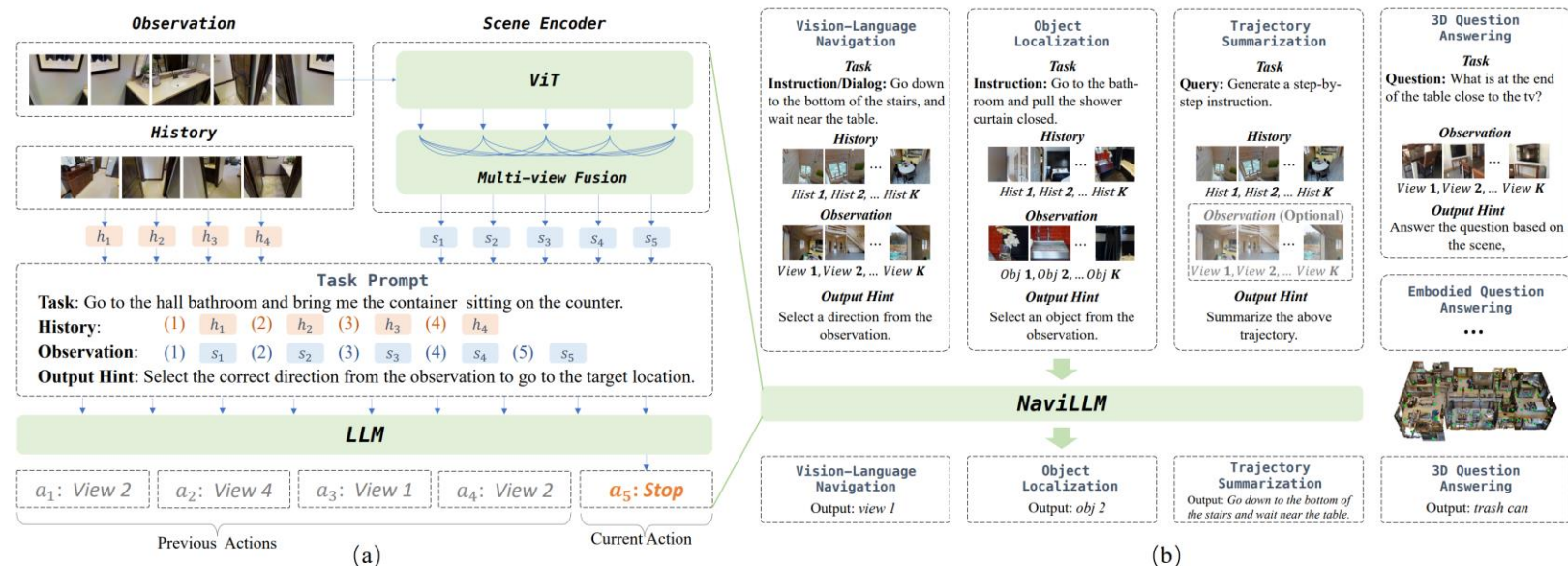




# End-to-End Robot Navigation

## Typical VLN Model:

GPT-Navigator:  
Extend Large Language Models (LLMs) from simple text processing to embodied AI applications that can understand and navigate the physical world.



Source: D. Zheng, S. Huang, L. Zhao, Y. Zhong, and L. Wang, "Towards learning a generalist model for embodied navigation," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 13 624–13 634.

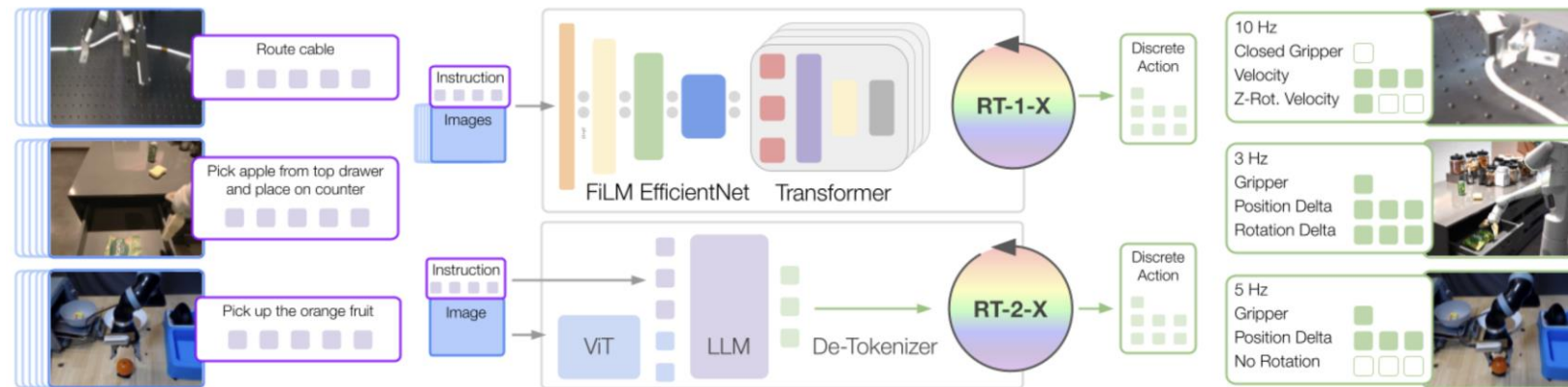
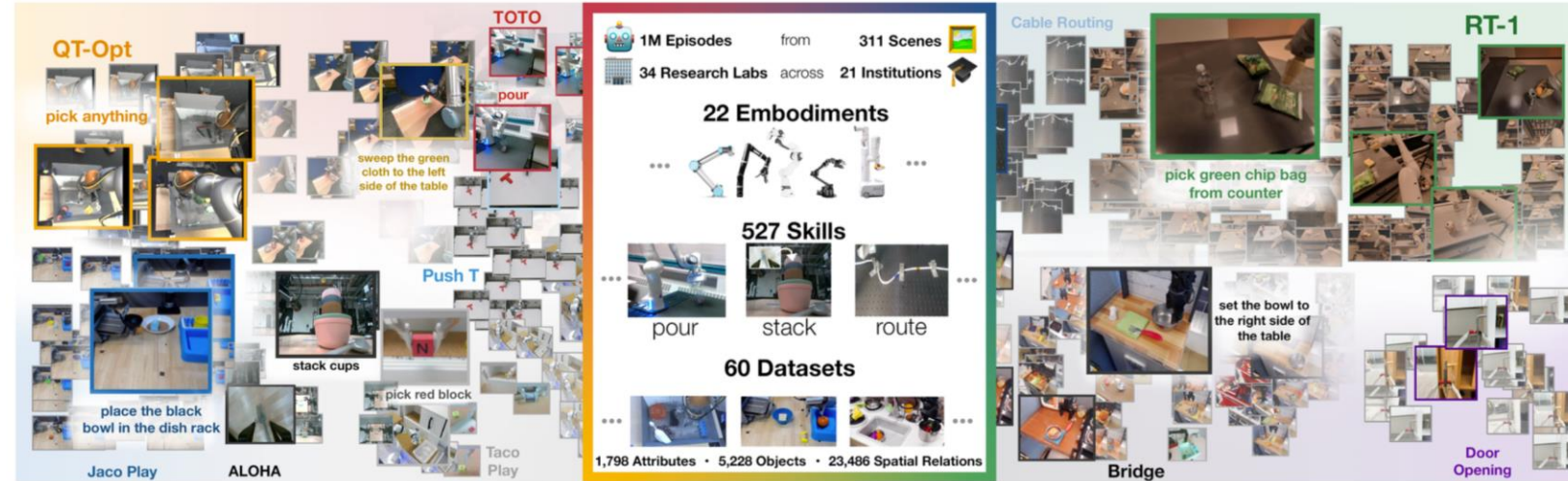
# Generalist Robot Models

## Trends:

- Single model → many robots/tasks
- Action tokens for robots
- Diffusion-based trajectory planners
- LLM as policy & skill routing

## Examples:

- Open-X Embodiment
- RT-X





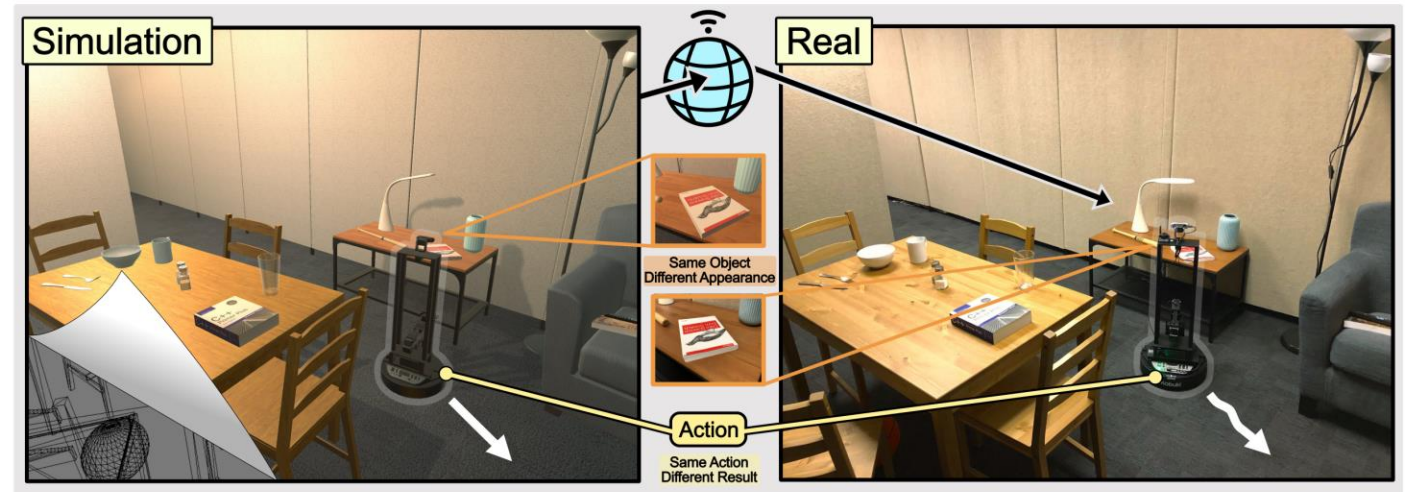
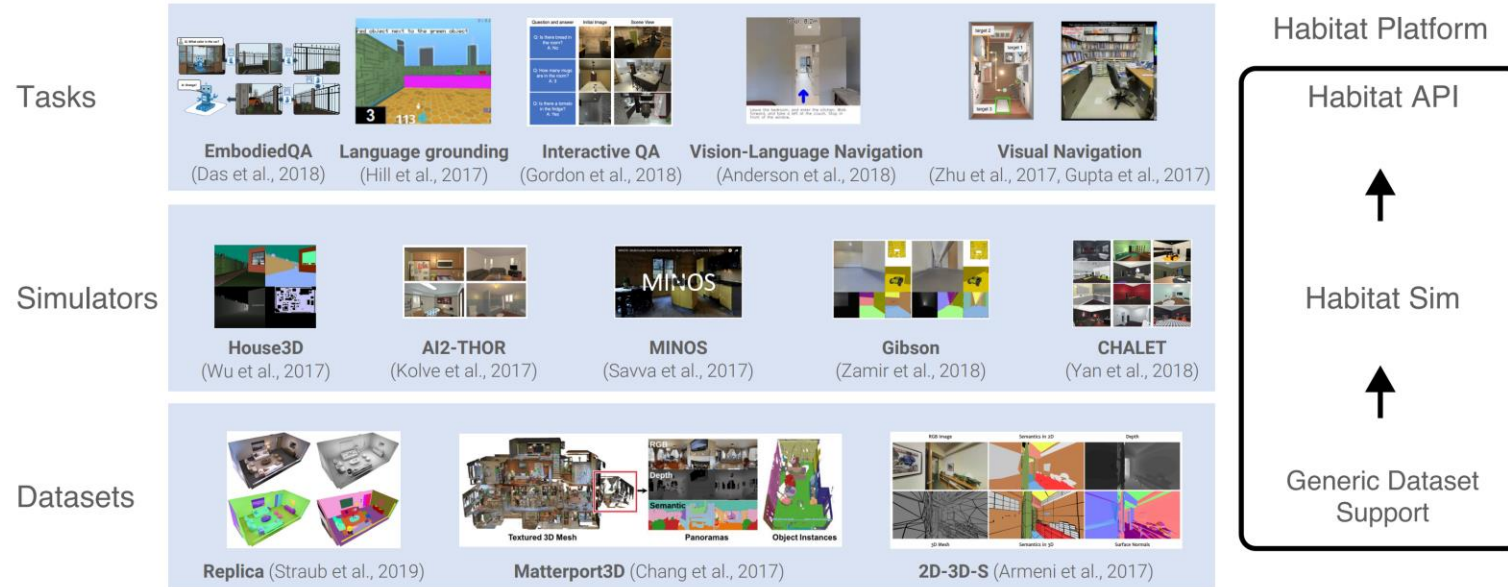
# Simulation Platforms

## Habitat:

A flexible, high-performance 3D simulator with configurable agents, multiple sensors, and generic 3D dataset handling

## RoboTHOR:

RoboTHOR is an environment within the AI2-THOR framework, designed to develop embodied AI agents. It consists of a series of scenes in simulation with counterparts in the physical world.



# Simulation Platforms

## Isaac Sim:

NVIDIA Isaac Sim™ is an open-source reference application built on NVIDIA Omniverse™ that enables developers to simulate and test AI-driven robotics solutions in physically based virtual environments.





# Learning Paradigms for Navigation Models

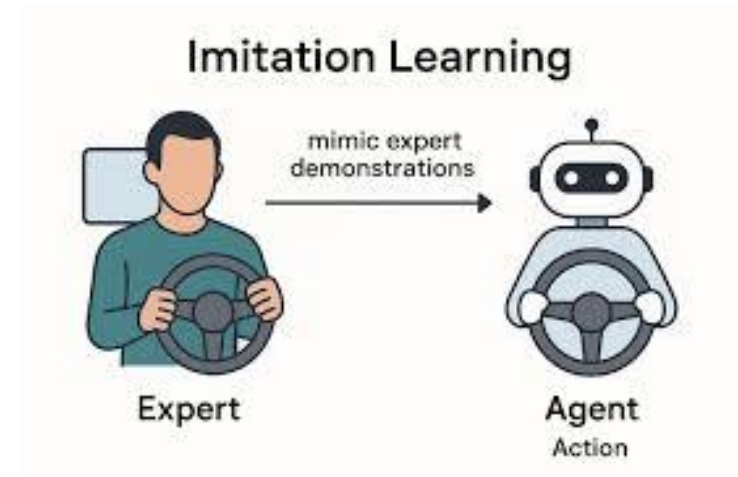
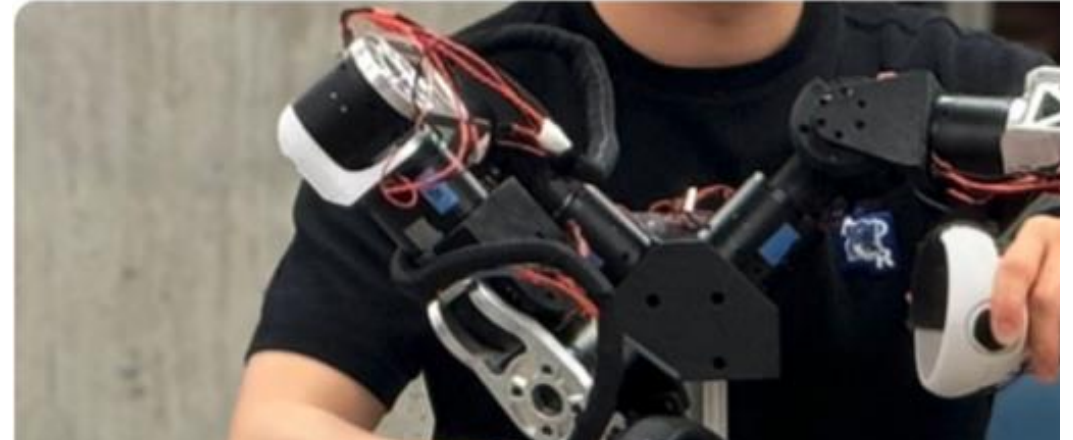
## Imitation Learning

**Behavioral Cloning (BC):** Mimics an expert's state-action mapping via supervised learning.

**Dataset Aggregation (DAGger):** Iteratively corrects BC's errors by querying the expert for labels on the agent's own visited states.

**Inverse Reinforcement Learning (IRL):** Infers the hidden reward function that explains the expert's behavior, rather than just mimicking actions.

**Adversarial Imitation Learning (AIL):** A discriminator network judges if a trajectory is from the expert or the agent, training the agent to deceive the discriminator.



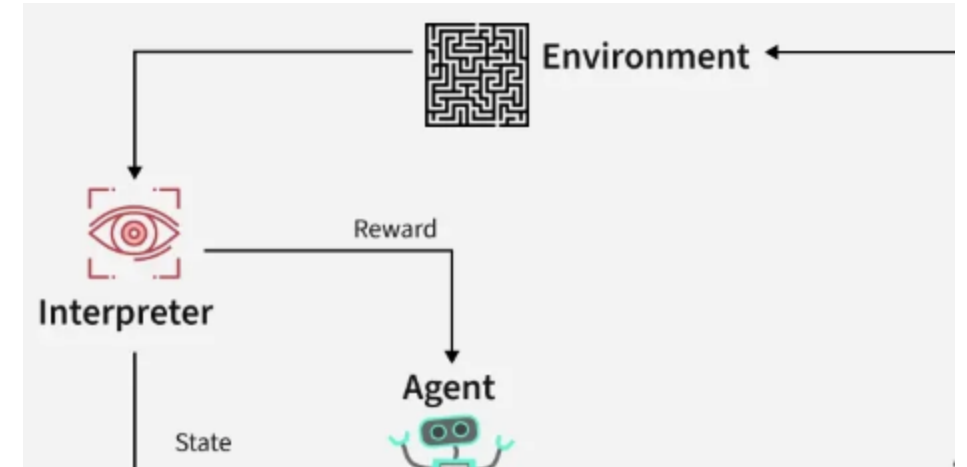
# Learning Paradigms for Navigation Models

## Reinforcement Learning:

Core Idea: An agent learns optimal actions through trial and error in an environment.

Mechanism: The agent maximizes a cumulative reward signal by discovering which behaviors yield the best long-term outcomes.

Analogy: Learning to navigate by exploration and receiving feedback (rewards/punishments).





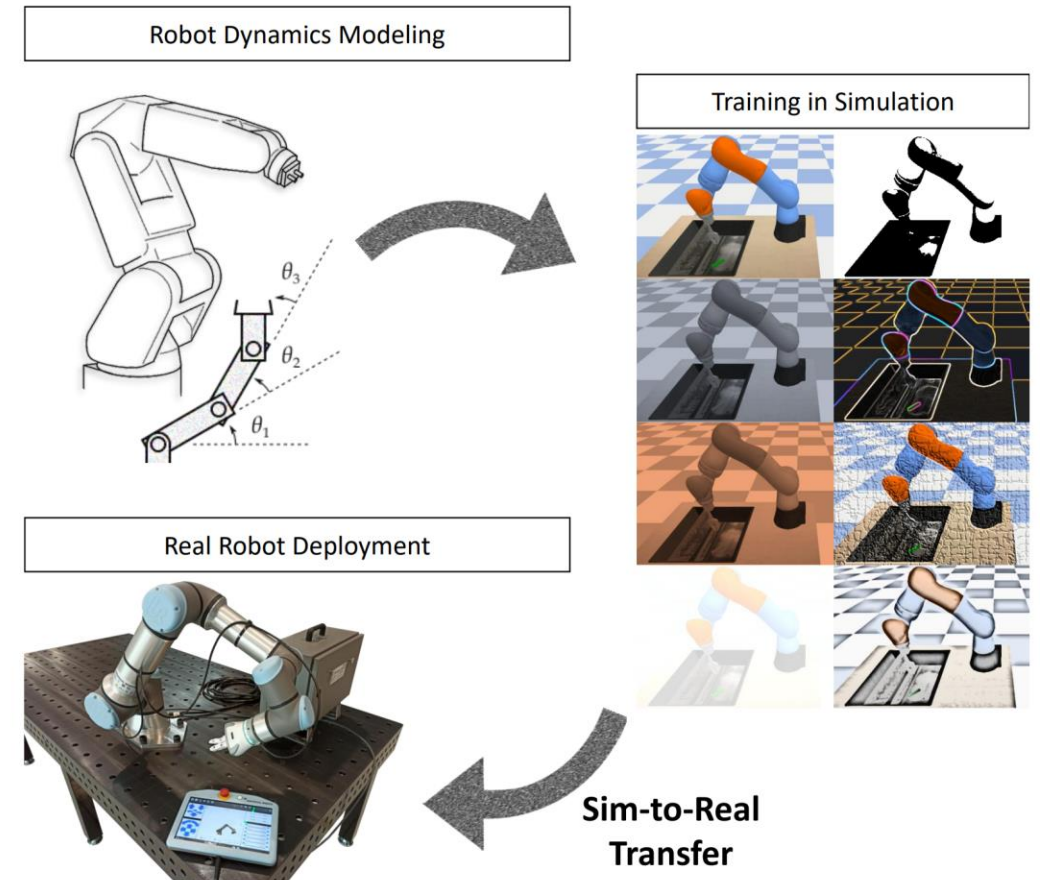
# Learning Paradigms for Navigation Models

## Transfer Learning

Core Idea: Leverage pre-existing knowledge from a source task to accelerate learning and improve performance on a new, target task.

Benefit: Avoids "starting from scratch," saving significant compute resources and data requirements.

Analogy: A professional racer using their skills to quickly master a new type of vehicle.



# New Trends and Discussion

## Open Questions

### **1. The Architectural Dilemma**

What is the optimal balance between interpretable modular systems and end-to-end models?

### **2. The Scalability vs. Efficiency Trade-off**

How can we develop end-to-end models that are both large enough to be capable and small enough for real-time deployment?

### **3. Guaranteeing Safety & Alignment**

How do we provably integrate hard safety constraints and ethical rules into data-driven learning systems?

# New Trends and Discussion

## Key Challenges in Embodied AI

- Sim-to-real transfer
- Long-horizon memory
- 3D reasoning + object permanence
- Social navigation
- Safety + human comfort
- Efficient data collection

## Summary

- Self-driving → hybrid E2E stacks w/ BEV transformers
- Embodied AI → multimodal learning (vision-language-action)
- World models & robot foundation models emerging
- Safety, reliability, grounding remain hardest problems





# Thanks for your attention!

Changhao Chen  
HKUST (GZ)

[changhaochen@hkust-gz.edu.cn](mailto:changhaochen@hkust-gz.edu.cn)

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)